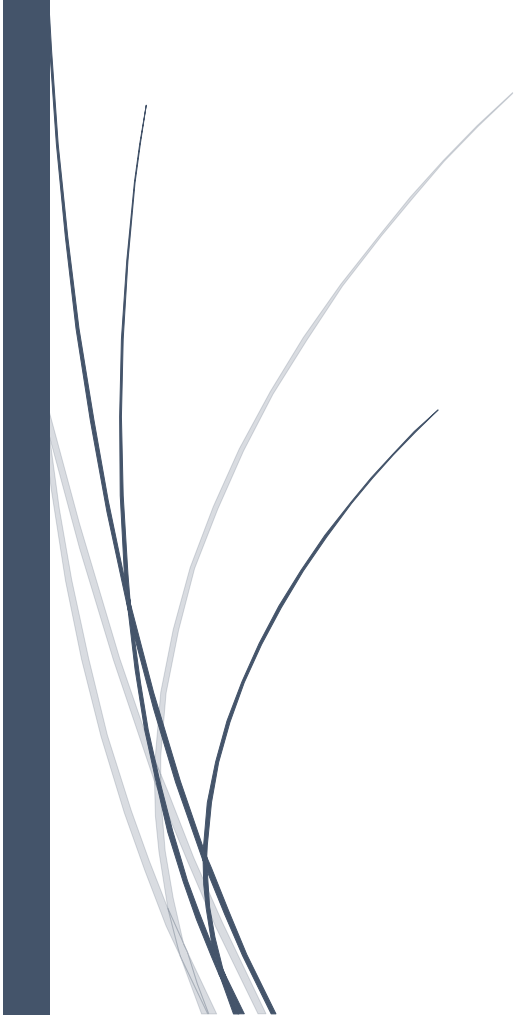
UT7. Persistencia en BBDD

CFGS

Desarrollo de Aplicaciones Multiplataforma



Dpto. Informática

IES AUGUSTÓBRIGA (NAVALMORAL DE LA MATA)

Contenido

1. Introducción. 2
2. Funciones básicas de los conectores....2
3. Capa de abstracción de Bases de Datos en Python. La librería *pydal* 3
4. Bases de datos relacionales. 7
4.1. SQLite y Python (sqlite3) 7
4.2 Diferentes bases de datos profesionales. 18
5. Mapeo de objetos relacionales – ORM....18
6. Bases de datos NoSQL.....19
7. Sistemas de Clave-Valor y Cachés en Python 20

1. Introducción.

Como ya sabemos, las bases de datos representan una de las mejores formas de almacenar la información y además, tienen la particularidad de que esa información puede ser consultada/modificada en el futuro de forma rápida y sencilla.

En los sistemas de bases de datos existen dos bloques diferenciados: las bd SQL y las bd no-SQL. Dentro de cada uno de estos bloques existen multitud de bases de datos diferentes, con distintas finalidades y para las que Python cuenta con herramientas que nos permiten interactuar con ellas. A diferencia de los ficheros, que se encuentran de forma local en la misma máquina donde se ejecuta el código, las bases de datos se encuentran (por norma general) en su propio servidor, contenedor, etc., lo que permite que distintas aplicaciones puedan conectarse a la misma base de datos.

Como no podía ser de otra forma, hoy en día nos encontramos con bases de datos alojadas en plataformas en la nube tales como AWS, Oracle Cloud, Microsoft Azure, IBM Cloud o Google Cloud, entre muchas otras. Como podemos suponer, todo esto supone que "necesitamos" una forma o mecanismo común para conectar Python con los distintos tipos de Bases de Datos. Con el fin de tener **una API común, se creó la PEP-249**, que recoge los métodos y características que deben tener los conectores de cualquier base de datos y que describimos en los siguientes puntos.

2. Funciones básicas de los conectores

La función básica para realizar una conexión a BD es **connect** y se define como se muestra a continuación:

- connect(fuente_datos='host:db_name', user='usuario', password='clave'). ¿Qué significa esto?, pues que sencillamente, Python obliga a todas las librerías que "utilicemos" para conectar con nuestra base de datos a realizar la conexión de esta forma. Esta función puede tener características particulares dependiendo del conector utilizado y la base de datos a utilizar (sqlite, postgresql, mysql…) pero siempre debe devolver un objeto tipo **Connection.**

Además, todas las instancias de la clase **Connection** que se creen al realizar la llamada a **connect** deben implementar una serie de funciones entre las que se encuentran las siguientes:

- **Connection.close()**: cierra la conexión abierta con la base de datos.
- **Connection.commit():** hace que los cambios pendientes de realizar en la base de datos desde que se realizó la conexión, se guarden de forma persistente. En muchos casos, esta operación no será necesaria si el conector de base de datos concreto está configurado para realizar el *commit* de forma automática.

- **Connection.rollback():** deshace los cambios que no se hayan guardado en la base de datos. Ojo, porque si existe el commit automático, los cambios no se podrán deshacer.
- **Connection.cursor():** devuelve una instancia de la clase **Cursor** que nos permite realizar operaciones con la base de datos. El Cursor será el encargado de trabajar con los datos de nuestra conexión.

Las instancias de la clase **Cursor,** es decir, el *cursor* devuelto por Connection.cursor(), implementan principalmente las siguientes funciones:

- **Cursor.callproc(nombre_procedimiento, *args):** permite ejecutar llamadas a procedimientos guardados en las bases de datos. Es opcional.
- **Cursor.close():** cierra el cursor asociado. Es más que recomendable que, siempre que se abre un cursor, este sea cerrado para evitar múltiples conexiones abiertas o bloqueos de tablas accedidas por distintos cursores abiertos.
- **Cursor.execute(sentencia, *args):** permite ejecutar cualquier sentencia soportada por la base de datos dentro del cursor asociado. Por ejemplo, una SELECT.
- **Cursor.executemany(sentencia, secuencia_parametros):** ejecuta la misma sentencia para un iterador de parámetros. Se utiliza para inserciones en serie de un iterador de parámetros a modo de filas.
- **Cursor.fetchone():** permite recibir el primer valor (y no todos) devuelto por la sentencia ejecutada en el cursor (execute).
- **Cursor.fetchmany(size=cursor.arraysize):** permite recibir una cantidad seleccionable de resultados definida por *Cursor.arraysize,* que es un atributo que define el número de líneas que debe leer el cursor en cada iteración y que por defecto es 1.
- **Cursor.fetchall():** devuelve todos los resultados sobre el cursor asociado.

Como ya se ha indicado, estas funciones son propias de la clase *Connection* y están definidas por la PEP-249 para servir de guía a los distintos conectores de las distintas bases de datos. Nosotros, dado que este módulo trata de ser una introducción y su cometido no trata de abordar en profundidad todos los tipos de bases de datos, nos centraremos principalmente en **SQLITE** como Base de Datos Relacional y en **DBM** como base de datos NoSQL (dado que se encuentra en el núcleo de Python)

3. Capa de abstracción de Bases de Datos en Python. La librería *pydal*

Antes de abordar las bases de datos relacionales, es necesario mencionar la librería **pydal.** Esta librería pretende hacer más fácil la integración de Python con cualquier base de datos mediante una mínima capa de abstracción.

Pydal permite las conexiones con las bases de datos, la definición de tablas, inserción de elementos, uniones de tablas, búsquedas agregadas y búsquedas básicas o anidadas sin depender de cada conector de la base de datos que estemos tratando.

Es muy importante tener claro que esta librería nos servirá para realizar operaciones básicas. Si se necesita realizar operaciones más específicas sobre cada base de datos, se recomienda utilizar el conector específico. Las bases de datos soportadas por esta librería, entre otras, son: SQLite, MySQL, PostgreSQL, FireBird, Oracle, MongoDB…

Veamos el siguiente ejemplo:

Para poder realizar este ejemplo desde VSC, es necesario abrir un terminal en Visual y lanzar la siguiente sentencia que instalará **pydal** (no viene instalada en el núcleo de Python)

- **pip install pydal**

Ahora vamos a probar el siguiente ejemplo. Crea un fichero tablas.py

tablas.py

```
#Creación de tablas para la base de datos
import datetime
from pydal import Field

#Función para crear las tablas en la base de datos
def crear_tablas(db):
    try:
        db.perfiles.drop()
        db.usuarios.drop()
        db.posts.drop()
    except Exception:
        print("No se pudieron eliminar las tablas, es posible que no existan.")
    #Tabla de perfiles
    db.define_table('perfiles',
                    Field('id_perfil',type='id'),
                    Field('nombre_perfil', type='string'),
                    Field('puede_editar',type='boolean')
                    )
    #Tabla de usuarios
    db.define_table('usuarios',
                    Field('id_usuario',type='id'),
                    Field('nombre_usuario', type='string'),
                    Field('apellido', type='string'),
                    Field('fecha_nacimiento', type='date'),
                    Field('perfil_id', type='reference perfiles')
                    )
    #Tabla de posts
    db.define_table('posts',
                    Field('id_post',type='id'),
                    Field('autor_id', type='integer'),
                    Field('texto', type='string'),
```

```
                Field('fecha', type='date'),
                Field('likes', type='integer')
                )


#función para insertar datos de ejemplo
def insertar_datos(db):
    db.usuarios.truncate()
    db.perfiles.truncate()
    db.posts.truncate()

    #Insertar perfiles
    db.perfiles.insert(nombre_perfil='Administrador', puede_editar=True)
    db.perfiles.insert(nombre_perfil='Visitante', puede_editar=False)
    db.perfiles.insert(nombre_perfil='Editor', puede_editar=True)

    #Insertar usuarios
    db.usuarios.insert(nombre_usuario='Juan', apellido='Pérez',
fecha_nacimiento=datetime.datetime(1990, 5, 15), perfil_id=3)
    db.usuarios.insert(nombre_usuario='María', apellido='Gómez',
fecha_nacimiento=datetime.datetime(1985, 8, 22), perfil_id=1)
    db.usuarios.insert(nombre_usuario='Carlos', apellido='López',
fecha_nacimiento=datetime.datetime(1992, 3, 10), perfil_id=3)

    #Insertar posts
    db.posts.insert(autor_id=1, texto='¡Hola a todos!',
fecha=datetime.datetime(2020,10,28), likes=10)
    db.posts.insert(autor_id=1, texto='¡Bienvenidos a mi blog!',
fecha=datetime.datetime.now(), likes=1)
    db.posts.insert(autor_id=2, texto='¿Cómo hacer ejercicio?',
fecha=datetime.datetime(2020,9,8), likes=40)
    db.posts.insert(autor_id=2, texto='¡Dietas saludables!',
fecha=datetime.datetime(2020,7,19), likes=15)

    #guardar cambios
    db.commit()
```

Ahora creamos el fichero principal, consultas.py

consultas.py

```
import tablas as t
from pydal import DAL

def main():
    # preparamos la conexión a la base de datos
    # Utilizamos SQLite para este ejemplo porque es fácil de conectar y no requiere
configuración adicional. SQLite
    # es un sistema de gestión de bases de datos ligero que almacena los datos en un
archivo local.
    # Es ideal para aplicaciones pequeñas o para desarrollo y pruebas.
    # A continuación, se muestra cómo conectar a una base de datos SQLite utilizando
PyDAL

    db = DAL('sqlite://ejemplo_pydal.db')

    # Una vez conectados, creamos las tablas y los datos de ejemplo

    t.crear_tablas(db)
    t.insertar_datos(db)

    # Ahora podemos realizar consultas a la base de datos utilizando PyDAL. A
continuación, se muestran algunos ejemplos de consultas:

    # Vamos a realizar una consulta que nos devuelva los usuarios cuyo apellido
comience con 'L'

    # primero vamos a definir la consulta
    consulta = db.usuarios.apellido.startswith('L')
    # luego ejecutamos la consulta y obtenemos los resultados
    resultados = db(consulta).select()
    # finalmente, imprimimos los resultados
    for usuario in resultados:
        print(f"Nombre: {usuario.nombre_usuario} {usuario.apellido}, Fecha de
Nacimiento: {usuario.fecha_nacimiento}, Perfil ID: {usuario.perfil_id}")

if __name__ == "__main__":
    main()
```

Prueba a modificar el código para añadir alguna consulta más como por ejemplo, que te muestre información de aquellos posts con más de 10 likes.

4. Bases de datos relacionales.

La principal característica de una base de datos relacional es que su contenido guarda relación entre sí. Este contenido, por regla general, se guarda en múltiples tablas y estas tablas se enlazan entre sí mediante uniones (JOIN) que representan a nivel lógico como se relaciona el contenido de unas tablas con otras.

No es objetivo de este capítulo explicar y/o profundizar en el conocimiento de las bases de datos relacionales dado que, en este punto, se entiende que los conceptos acerca del modelo relacional están superados.

Por ello, omitiremos cualquier referencia al modelo relacional y su estructura así como al lenguaje de consultas SQL necesario para realizar la definición (DDL) y la manipulación (DML) de los datos.

4.1. SQLite y Python (sqlite3)

La base de datos relacional más simple y más utilizada es SQLite, dado que es una base de datos pequeña, rápida, auto contenida y muy fiable. Una gran ventaja es que permite la integración con muchos lenguajes de programación de forma casi nativa. Quizás, el principal motivo que la hace ser tan utilizada es que se usa en dispositivos móviles casi por defecto, permitiendo guardar de forma ordenada pequeñas cantidades de datos.

Seguramente, la principal característica de esta base de datos es que no necesita estructura cliente-servidor sino que la propia base de datos se integra en al aplicación. Por el contrario, al ser tan pequeña, ofrece menos características que bases de datos más grandes.

En Python, la librería que nos permite interactuar con bases de datos SQLite está integrada en el núcleo de Python y se llama **sqlite3.** Esta librería permite la interacción usando la API de bases de datos vista en el **punto 2** con matices y extensiones que vemos a continuación:

- **sqlite3.connect(database[, timeout, detect_types, isolation_level, check_same_thread, factory, cached_statements, uri]):** permite crear una conexión con la base de datos y devuelve por defecto un objeto tipo *Connection.*
  - ***database****:* define la ruta de un objeto o se puede utilizar un parámetro especial con valor **:memory:** que permite crear una base de datos en memoria en lugar de disco (lo que hace que sea ultra rápida pero, ojo, se borra al cerrar el programa)
  - ***timeout***: permite seleccionar los segundos que una consulta tarda en devolver un error mientras la base de datos está bloqueada (algo ya estaba accediendo a la base de datos). Esto permite o da tiempo a que la base de datos se desbloquee. Por defecto es 5 seg.
  - ***detect_types***: permite especificar si se deberían detectar los tipos devueltos de las consultas. Tiene dos modos:

detect_types=sqlite3.PARSE_DECLTYPES | sqlite3.PARSE_COLNAMES

sqlite3.PARSE_DECLTYPES: Mira el tipo que pusiste al crear la tabla (CREATE TABLE).
Si definiste la columna como fecha TIMESTAMP, cuando hagas un SELECT, Python verá la palabra "TIMESTAMP" y convertirá el texto automáticamente a un objeto datetime.datetime.

sqlite3.PARSE_COLNAMES: Mira el nombre que le das a la columna en la propia consulta SQL usando un alias.
Ejemplo: SELECT fecha AS "f [datetime]" FROM tabla. Aquí le estás "chivando" a Python el tipo de dato en tiempo de ejecución.

- ***uri:*** si el valor de *database* es una URI que identifica la base de datos, se debe indicar el valor True aquí.

- **sqlite3.register_converter(typename, callable):** permite definir cómo convertir un objeto tipo *bytestring* devuelto por las consultas de la base de datos en objetos Python.
  Tenemos que ver que SQLite trabaja con datos muy concretos.

  En realidad, SQLite solo trabaja con 5 tipos internos:

Tipo SQLite	Qué representa
`NULL`	Valor nulo
`INTEGER`	Enteros
`REAL`	Números decimales
`TEXT`	Cadenas
`BLOB`	Datos binarios

Todo lo demás (DATE, BOOLEAN, etc.) es “decorativo”.

Ejemplo con fechas

CREATE TABLE ejemplo (fecha DATE);
Aunque pongas DATE, SQLite puede guardar:

INSERT INTO ejemplo VALUES ('2026-04-15'); -- texto
INSERT INTO ejemplo VALUES (1713139200); -- entero (timestamp)
INSERT INTO ejemplo VALUES (NULL); -- null

Todo es válido.

Por tanto, si hemos insertado un dato en la base de datos SQLite que queremos representar como una fecha en Python, necesitamos hacer uso de esta función y siempre y cuando hayamos aplicado en el parámetro ***detect_types*** de **connect** el valor de PARSE_DECLTYPES.

Ejemplo:

```
import sqlite3
from datetime import datetime


def convertir_fecha(valor):
    return datetime.strptime(valor.decode(), "%Y-%m-%d")


sqlite3.register_converter("DATE", convertir_fecha)


conn = sqlite3.connect("mi_base.db", detect_types=sqlite3.PARSE_DECLTYPES)
cursor = conn.cursor()
# la siguiente sentencia realmente no va a crear ningún tipo DATE en la base
# de datos porque no existe ese tipo. Simplemente nos permitirá utilizar
# un campo como un decorador que luego podemos convertir en fecha
# fíjate que lo que se inserta es una cadena...
cursor.execute("CREATE TABLE ejemplo (fecha DATE)")
cursor.execute("INSERT INTO ejemplo VALUES (?)", ("2026-04-15",))


# en este momento, al hacer la consulta, el módulo sqlite3 detecta que tienes un
# campo DATE sobre el que has definido la función register_converter y “hace su
```

```
# trabajo llamando a la función que has definido para convertir la cadena en un
# tipo fecha.
cursor.execute("SELECT fecha FROM ejemplo")
resultado = cursor.fetchone()
print(type(resultado[0]))  # <class 'datetime.datetime'>
```

- **sqlite3.register_adapter(type, callable):** permite definir cómo convertir objetos Python en objetos soportados por SQLite, que deben ser uno de los siguientes dado que, como hemos visto antes, SQLite sólo acepta un rango determinado de tipos: int, float, str o bytes (tipos Python). Mira el siguiente ejemplo:

```
import sqlite3

# 1. Definimos nuestra clase de Python
class ListaCompra:
    def __init__(self, productos):
        self.productos = productos  # Es una lista: ["pan", "agua"…]

# 2. Definimos el 'callable' (el adaptador)
# Recibe el objeto Python y devuelve lo que irá a la BD
def adaptar_lista(lista_obj):
    return ",".join(lista_obj.productos)

# 3. Registramos el adaptador
# (Clase de Python, Función que adapta)
sqlite3.register_adapter(ListaCompra, adaptar_lista)

# --- PRUEBA DEL FUNCIONAMIENTO ---

con = sqlite3.connect(":memory:") # Creamos una base de datos en memoria para
pruebas
cur = con.cursor()
cur.execute("CREATE TABLE pedidos (id INTEGER, contenido TEXT)")

# Creamos el objeto Python
mi_lista = ListaCompra(["Manzanas", "Leche", "Huevos"])

# Al insertar, pasamos el objeto directamente.
# SQLite llamará a nuestro adaptador automáticamente.
cur.execute("INSERT INTO pedidos VALUES (?, ?)", (1, mi_lista))
cur.execute("INSERT INTO pedidos VALUES (?, ?)", (2, ListaCompra(["Pan",
"Agua"])))

# Verificamos qué se ha guardado físicamente
```

```
cur.execute("SELECT id, contenido FROM pedidos")
for fila in cur.fetchall():
    print(f"En la base de datos se ha guardado el pedido con id {fila[0]}:
{fila[1]}")
```

Observa que, como “cosas” a observar, utilizamos la opción :memory: para guardar la base de datos en memoria, por lo que no se genere ningún fichero de base de datos en tu disco.

Además, como hemos realizado la inserción de 2 registros, utilizamos un **for** para recorrer el **cursor (cur)** e imprimir los resultados de la consulta.

- **sqlite3.complete_statement(sql):** verifica si el parámetro pasado es una sentencia completa, es decir, si tu sentencia termina con ‘;’. Cuidado porque no sirve para comprobar si la sentencia es correcto; de eso ya se encarga slite3.execute(sentencia).

- **Connection.cursor(factory=Cursor):** genera cursores para poder interactuar con la base de datos. Por defecto genera cursores tipo Cursor pero se puede cambiar usando factory.

  ¿Y qué es el Cursor por defecto?
  Cuando haces Connection.cursor(), Python te da un objeto que devuelve los resultados como tuplas. Y esto es más rápido y consume poca memoria pero tiene la desventaja de que debes recordar que la columna 0 es, por ejemplo, el ID, la 1 es el Nombre, etc.

  El parámetro factory permite “cambiar” el molde. El uso más común es usar **sqlite3.Row**. Aunque no es técnicamente un "factory" de cursor (se suele asignar a **con.row_factory**), el concepto es el mismo: cambiar cómo se generan los resultados. Ejemplo:

```
import sqlite3

con = sqlite3.connect(":memory:")
# CONFIGURACIÓN CLAVE:
con.row_factory = sqlite3.Row

cur = con.cursor()
cur.execute("CREATE TABLE usuarios (id INTEGER, nombre TEXT)")
cur.execute("INSERT INTO usuarios VALUES (1, 'Alberto')")

cur.execute("SELECT * FROM usuarios")
fila = cur.fetchone()

# Ahora puedes hacer esto:
```

```
print(fila['nombre'])  # Salida: Alberto (¡Mucho más legible!)
print(fila[1])         # Salida: Alberto (Sigue funcionando por índice)
```

Nos permite trabajar con los nombre de las columnas en lugar de recordar los índices de los campos.

- **Connection.commit():** guarda los cambios pendientes de forma persistente.
- **Connection.rollback():** deshace los cambios pendientes.
- **Connection.close():** cierra la conexión con la base de datos.
- **Connection.create_function(nombre, num_params, func):** permite enlazar una función escrita en Python para ejecutarla en una consulta SQL

```
import sqlite3

#definimos la función

def doble(x):

    return x * 2


conn = sqlite3.connect(":memory:")

conn.create_function("mi_funcion", 1, doble)

cursor = conn.cursor()

cursor.execute("SELECT mi_funcion(5)")

print(cursor.fetchone())  # (10,)
```

- **Connection.create_aggregate(name, num_params, aggregate_class):** permite enlazar una función de Python que agregue resultados sobre una serie de valores que se pasarán en la consulta SQL. Permite definir una clase de Python para que puedas utilizarla como si fuera una función de agregación (SUM, AVG, etc) definida por ti. Es necesario que la clase defina dos funciones: *step* y *finalize.* La función *step* se encarga de actualizar el valor final con cada valor nuevo, y *finalize* devuelve el resultado final que debe ser del tipo permitido por SQLite (int, str, float, byte o None)

```
import sqlite3

class Producto:

  # inicializamos
  def __init__(self):
     self.resultado = 1

  # con esta función, actualizamos el valor final con cada valor nuevo
  # recibido en la consulta…
  def step(self, value):
     self.resultado *= value

   # devolvemos el resultado
   def finalize(self):
     return self.resultado

conn = sqlite3.connect(":memory:")
conn.create_aggregate("producto", 1, Producto)

cursor = conn.cursor()
cursor.execute("""
    SELECT producto(valor)
    FROM (SELECT 2 AS valor UNION ALL SELECT 3 UNION ALL SELECT 4)
    """)

print(cursor.fetchone()[0]) # 24
```

Debemos destacar que cada **cursor** es el encargado de ejecutar los comandos que se deseen y que se pueden tener muchos cursores en una sola conexión. A continuación describimos las funciones más utilizadas por los cursores:

- **Cursor.execute(sql [, parametros]):** permite la ejecución de cualquier sentencia SQL con o sin parámetros. Los parámetros se pueden especificar mediante interrogaciones (?) y una tupla de valores o con marcadores (:param1, :color, :val2, etc) y un diccionario en el que las claves son esos marcadores y los valores son el valor a usar en el parámetro.
- **Cursor.executemany(sql, sec_de_parametros):** permite ejecutar la misma sentencia SQL con una secuencia de parámetros. Es muy utilizado para realizar la inserción de varias filas. En la sentencia se define el comando INSERT con los parámetros requeridos y se inserta una secuencia de tuplas con los valores. Otra opción es utilizar un iterador escrito en Python como secuencia de parámetros.

- **Cursor.executescript(sql_script):** función no estándar que permite la ejecución de un script completo en SQL con múltiples sentencias. Al inicio ejecuta un COMMIT y luego va ejecutando cada sentencia como una única cadena de caracteres.
- **Cursor.fetchone():** permite recibir el siguiente valor de la consulta realizada en el cursor. Si no queda más información devuelve **None**.
- **Cursor.fetchmany(size=cursor.arraysize):** permite recibir el siguiente bloque de información; el número de filas lo define el parámetro size. Cuando no quedan más elementos que devolver devuelve una lista vacía.
- **Cursor.fetchall():** permite recibir todo el contenido que quede pendiente en el cursor en forma de lista de elementos.
- **Cursor.close():** permite cerrar el cursor asociado. A partir de aquí, si se intenta utilizar el cursor se devuelve una excepción del tipo *ProgrammingError.*

Tenemos los siguientes atributos:

- **Cursor.rowcount**: si la consulta “lo permite”, devuelve el número de filas afectadas por los comandos ejecutados en el cursor.
- **Cursor.description:** es un atributo de solo lectura que permite obtener el nombre de las columnas de los valores devueltos por la última consulta realizada en el cursor.

Estudia el siguiente ejemplo donde utilizamos lo visto anteriormente…

```
import sqlite3

# 1. Crear conexión (en memoria)
conn = sqlite3.connect("ejemplo_sqlite3.db")  # Esto crea un archivo en el disco con
la base de datos
cursor = conn.cursor()

# antes de nada, vamos a eliminar las tablas si ya existen para evitar errores
try:
    cursor.execute("DROP TABLE IF EXISTS usuarios")
    cursor.execute("DROP TABLE IF EXISTS perfiles")
    cursor.execute("DROP TABLE IF EXISTS posts")
except Exception as e:
    print("Error al eliminar tablas (puede que no existan):", e)

# 2. Crear tablas
cursor.execute("""
CREATE TABLE usuarios (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    nombre TEXT,
    apellidos TEXT,
    fecha_nacimiento TEXT,
```

```
    perfil_id INTEGER,
    FOREIGN KEY(perfil_id) REFERENCES perfiles(id)
)
""")

cursor.execute("""
CREATE TABLE perfiles (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    nombre_perfil TEXT,
    puede_editar TEXT
)
""")

cursor.execute("""
CREATE TABLE posts (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    autor_id INTEGER,
    texto TEXT,
    fecha TEXT,
    likes INTEGER,
    FOREIGN KEY(autor_id) REFERENCES usuarios(id)
)
""")

# 3. Insertar en la tabla usuarios
usuarios = [
    ("Luis", "López"),
    ("Ana", "Martínez"),
    ("Laura", "Luna"),
    ("Carlos", "Pérez"),
    ("Lucía", "Lara")
]

# Usamos executemany para insertar varias filas de una vez mediante una lista de tu-
plas
cursor.executemany("INSERT INTO usuarios (nombre, apellidos) VALUES (?, ?)", usua-
rios)

# 4. Insertar en la tabla perfiles
perfiles = [
    ("Administrador", "Sí"),
    ("Editor", "Sí"),
    ("Visitante", "No")
]

# Usamos executemany para insertar varias filas de una vez mediante una lista de tu-
plas
```

```
cursor.executemany("INSERT INTO perfiles (nombre_perfil, puede_editar) VALUES (?, ?)", perfiles)

# Usamos executemany para insertar varias filas de una vez mediante una lista de tuplas
cursor.executemany("INSERT INTO usuarios (nombre, apellidos) VALUES (?, ?)", usuarios)

# 5. Insertar en la tabla posts
posts = [
    (1, "Post de Luis 1", "2024-01-01", 10),
    (1, "Post de Luis 2", "2024-01-02", 3),
    (2, "Post de Ana", "2024-01-03", 7),
    (3, "Post de Laura", "2024-01-04", 2),
    (5, "Post de Lucía", "2024-01-05", 8),
    (4, "Post de Carlos", "2024-01-06", 1),
    (5, "Otro post de Lucía", "2024-01-07", 12),
    (2, "Otro post de Ana", "2024-01-08", 4)

]

# Usamos executemany para insertar varias filas de una vez mediante una lista de tuplas
cursor.executemany("INSERT INTO posts (autor_id, texto, fecha, likes) VALUES (?, ?, ?, ?)", posts)

# Guardamos los cambios
conn.commit()

# ------------------------------
# 6. CONSULTA 1:
# Usuarios cuyo apellido empieza por L
# ------------------------------
print("Usuarios con apellido que empieza por L:")
cursor.execute("""
SELECT DISTINCT nombre, apellidos
FROM usuarios
WHERE apellidos LIKE 'L%'
""")

for row in cursor.fetchall():
    print(row)

# ------------------------------
# 7. CONSULTA 2:
# Usuarios que han escrito posts con más de 5 likes
# ------------------------------
print("\nUsuarios que han escrito posts con más de 5 likes:")
```

```
cursor.execute("""
SELECT p.texto, u.nombre, u.apellidos, p.likes
FROM posts p
JOIN usuarios u ON p.autor_id = u.id
WHERE p.likes > 5
""")

for row in cursor.fetchall():
    print(row)

# Cerrar conexión
conn.close()
```

Sqlite3 posee un objeto muy particular llamado **Row** que puede ser utilizado para mapear la información que se obtiene de las consultas de manera que podemos acceder al valor de cada columna de cada fila. Prueba a realizar lo siguiente una vez que hayas ejecutado el ejemplo anterior y tengas creada la base de datos "**ejemplo_sqlite3.db":**

prueba_row.py

```
import sqlite3

conn=sqlite3.connect("ejemplo_sqlite3.db")
conn.row_factory = sqlite3.Row  # Esto hace que las filas se devuelvan como objetos
tipo diccionario
cursor=conn.cursor()
cursor.execute("SELECT * FROM usuarios")
# Obtenemos la primera fila del resultado
primer_usuario = cursor.fetchone()
# Imprime las columnas disponibles
print(f'{primer_usuario.keys()}')
# Accedemos a los datos por nombre de columna
print(f'{primer_usuario["id"]} {primer_usuario["nombre"]} {primer_usuario["apelli-
dos"]} {primer_usuario["fecha_nacimiento"]} {primer_usuario["perfil_id"]}')
```

Por supuesto, y aunque la finalidad es ser utilizada para aplicaciones relativamente pequeñas, sqlite3 tiene mucho potencial y se pueden realizar operaciones realmente complejas y potentes con la base de datos SQLite. Puedes acceder a toda la información de esta librería en la documentación de Python (https://docs.python.org/3/library/sqlite3.html)

4.2 Diferentes bases de datos profesionales.

En esta unidad sólo vamos a ver la base de datos relacional SQLite dado que resulta imposible introducirnos en todas y cada una de las bases de datos profesionales que podemos encontrar y con las que se puede trabajar desde Python.

La mayoría de estas bases de datos pueden manejar grandes volúmenes de datos y por ello requieren de una arquitectura Cliente-Servidor. El mantenimiento de estas bases de datos requiere un conocimiento más avanzado y en muchas ocasiones incluso especializado en cada base de datos para poder sacarle todo el partido.

Se recomienda revisar a fondo la documentación de cada una de las bases de datos si vamos a hacer uso de ellas y poder sacar el mayor provecho. A continuación se relacionan las principales bases de datos que tienen soporte directo de Python:

- IBM DB2 (https://wiki.python.org/moin/DB2)
- Firebird (https://wiki.python.org/moin/Firebird)
- Informix (https://wiki.python.org/moin/Informix)
- Actian X – Ingres (https://wiki.python.org/moin/Ingres)
- MySQL (https://wiki.python.org/moin/MySQL)
- Oracle (https://wiki.python.org/moin/Oracle)
- PostgreSQL (https://wiki.python.org/moin/PostgreSQL)
- SAP MaxDB (https://wiki.python.org/moin/SAP%20DB)
- Microsoft SQL Server (https://wiki.python.org/moin/SQL%20Server)
- Microsoft Access (https://wiki.python.org/moin/Microsoft%20Access)

5. Mapeo de objetos relacionales – ORM

Cuando se trabaja con objetos en Python y se pretenden guardar en una base de datos, ya sea SQL o NoSQL, a menudo se intenta eliminar la complejidad para crear código pythónico que no tenga que realizar consultas en crudo a la base de datos, serializando y deserializando los objetos, con consultas específicas para cada base de datos o con migraciones si hay cambios en las tablas o la lógica de la aplicación. Para mitigar estos problemas se desarrollaron los **ORM** (mapeo de objetos relacionales).

Dado el carácter introductorio de este módulo, no vamos a profundizar en estos conceptos pero sí vamos a relacionar aquellos más importantes para que sea el propio alumno quién, llegado el momento, profundice para alcanzar una mayor conocimiento de los mismos.

Los ORM son una herramienta esencial cuando se trabaja con diferentes tipos de objetos (lo que ocurre normalmente cuando estamos desarrollando un proyecto relativamente grande), cuando los objetos tienen relaciones entre sí y cuando se pretenden realizar operaciones que resultarían en un sinfín de consultas a la base de datos que serían casi inmantenibles.

La mayoría de los ORM soportan la definición de los campos que debe tener cada tabla como si fueran atributos de una clase en Python y, además, ayudan a manejar los cambios que se puedan realizar en esa clase con una herramienta llamada “migraciones”. Cuando se crean las clases en Python, el ORM se encarga de crear y validar la estructura de la base de datos, y cuando se produce algún cambio, crea migraciones de la base de datos que hacen que todo el proceso sea mucho más mantenible.

Para que lo entendamos de una forma sencilla:

Convierte esto:

```
SELECT * FROM usuarios WHERE id = 1;
```

en algo así:

```
usuario = session.query(Usuario).filter_by(id=1).first()
```

Es decir:

- Tablas → clases
- Filas → objetos
- Columnas → atributos

A continuación relacionamos los ORM más populares utilizados en Python.

- SQLALCHEMY ORM
- PEEWEE ORM
- PONY ORM
- DJANGO MODELS
- ORM ASÍNCRONO GINO

6. Bases de datos NoSQL.

Una gran alternativa a las bases de datos relacionales son las denominadas NoSQL, las cuales rompen con algunas restricciones y paradigmas de las bases de datos relacionales.

Las bases de datos NoSQL apartan las normas establecidas por los modelos relacionales para favorecer otros aspectos como la velocidad de lectura o la opción de escalar horizontalmente.

Usar una base de datos NoSQL no significa que no haya soporte para realizar operaciones SQL, sino que no existen las restricciones ACID (Atomicidad, Consistencia, Aislamiento y Durabilidad). Dada esta flexibilidad, existen muchas opciones que se centran en un área específica de la computación, desde bases de datos orientadas a guardar solo elementos como clave-valor hasta bases de datos que guardan los datos de forma columnar, pasando por algunas que utilizan nodos y grafos para guardarlos.

A continuación relacionamos los distintos tipos de bases de datos y ejemplos de ellas:

- Bases de datos orientadas a documentos: MongoDB, DynamoDB, Cloud Firestore, Elasticsearch.
- Bases de datos en tiempo real: Firebase Realtime Database, RethinkDB
- Base de datos para series temporales: InfluxDB, TimescaleDB, OpenTSDB, Graphite.
- Bases de datos orientadas a grafos de datos: Neo4j, AWS Neptune, ArangoDB.
- Bases de datos columnares: AWS Redshift, MariaDB ColumnStore, Apache Hbase.
- Bases de datos para búsquedas de texto: Apache Lucen, Apache Solr, Elasticsearch.

7. Sistemas de Clave-Valor y Cachés en Python

Las estructuras de datos tipo clave-valor se usan principalmente para guardar datos de manera simple y ordenada. Siempre se tiene una clave única y un valor de un tipo determinado para cada elemento. Uno de los principales usos que se le da a este tipo de arquitecturas es la creación de **cachés**. Las cachés son herramientas que nos permite guardar la información de forma simple, rápida y disponible durante un tiempo. Transcurrido ese tiempo, se invalidan los datos y es necesario realizar una nueva carga desde el origen de los datos que normalmente es una base de datos con una consulta concreta. Por tanto, podemos definir los sistemas caché como “una capa creada por encima de la base de datos que mantiene las consultas que más nos interesan disponibles durante la mayoría del tiempo.

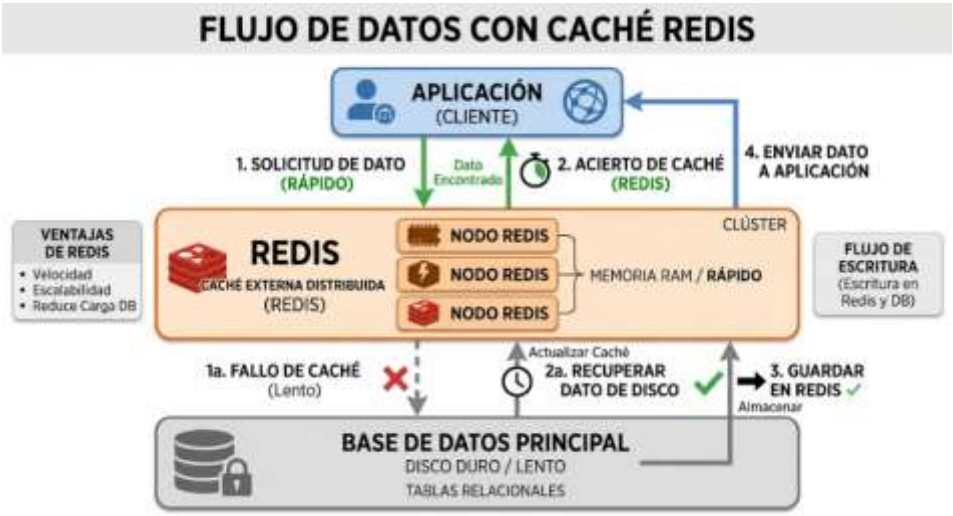


Figura 1 Arquitectura de un sistema caché con Redis

Una forma sencilla de crear una caché es haciendo uso de una base de datos NoSQL que guarde información de forma clave-valor. Vemos a continuación una pequeña introducción a los tres sistemas y librerías más conocidos para usar este tipo de bases de datos: **dbm, Memcached** y **Redis.**

- **DBM:** se encuentra en el núcleo de Python y es la base de datos basada en clave-valor más simple con la que trabajar. Guarda la información en disco utilizando la estructura clave-valor, soportando únicamente valores de tipo **str** o **bytes,** lo que hace que sea muy limitada.
- **MEMCACHED:** guarda los datos en memoria y no persiste la información en disco de manera que si se reinicia el servidor o el servicio, se perderá toda la información almacenada. Utiliza un algoritmo basado en las reglas LRU (least recently used; menos usado recientemente) de manera que la información que menos se ha usado recientemente es la primera que se elimina en el caso de necesitar más espacio de almacenamiento.
  La aplicación debe ser instalada en el servidor en el que se pretende usar el servicio, interactuando con él desde Python mediante la librería **pymemcache (**https://pymemcache.readthedocs.io/en/latest/). Se trabaja con esta librería mediante un cliente que se conectará a la máquina local, a un servidor diferente, a un servicio en al nube e incluso a más de un nodo de Memcached a la vez.
- **REDIS:** no es solo una base de datos utilizada para caché en memoria, sino que también tiene otras características, como la persistencia en disco (muy importante), la capacidad de trabajar en clústeres, el paso de mensajes, estructuras de datos diferentes, capacidad de realizar operaciones atómicas y otra serie de características que hacen que sea una base de datos muy popular y utilizada en el ámbito del desarrollo de aplicaciones. Quizás la característica a tener muy en cuenta es que tiene soporte de envío de mensajes, por lo que es muy utilizada para la creación de aplicaciones en las que se envíen mensajes utilizando la configuración de *publicador/subscriptor.*
  Además cuenta con un amplio abanico de tipos de datos soportados: Strings, Hashes, Lists, Sets, Sorted Sets, Bitmaps, HyperLogLogs, Geospatial indexes, Streams.
  Existen dos librerías principales en Python para trabajar con Redis: **redis-py** y **walrus.**
  (https://github.com/andymccurdy/redis-py)
  (https://walrus.readthedocs.io/en/latest/index.html)